

Studio 02: Context Window Stress Test & Memory Architecture — Step-by-Step Tutorial

COMS W4995-009 Agentic Engineering, Columbia University, Fall 2026 · Session 2 (Sep 18): What the Agent Can See, and What It Remembers

[At a glance](#)

[Before you start](#)

[Tutorial 1 — Set up the team repository and model access](#)

[Tutorial 2 — Run the three experiments](#)

[Tutorial 3 — Write up and submit](#)

[Wrap-up](#)

[Errata](#)

[Omitted](#)

At a glance

- **Goal:** Push a coding agent past its useful context length and watch it degrade; fix that with context isolation and targeted summarization, which cap peak single-call context tokens; then give the agent persistent file-based memory across separate sessions and compare it to a memoryless baseline.
- **You will produce:** evidence files from Parts A, B, and C plus a filled-in `EXPLANATION.md`, pushed to your team's private repository.
- **Time:** 75 minutes; teams of up to 3 students per team.
- **Where this fits:** Session 2 studio, Learning Objective 2 “Manage Context Infrastructure” (syllabus: Weekly Schedule, Learning Objectives).

Before you start

What you need

Requirement	Detail
Node.js	<code>>= 22.19</code> (required because the <code>pi</code> coding agent needs it)
<code>pi</code>	<code>npm install -g @earendil-works/pi-coding-agent</code>
Python	<code>>= 3.9</code> (see Errata: the course itself requires 3.11+)
GitHub CLI <code>gh</code>	logged in via <code>gh auth login</code>

Concepts you may not know

- **Global install** (`npm install -g`) — installs a command-line tool once for the whole machine and puts it on your PATH, instead of into one project's folder. Like `pip install` outside a virtualenv.
- **Environment variable** (`export NAME=value`) — a named value the shell hands to every program it starts. It lives only in that terminal session, so secrets stay off disk and out of git.
- **Token and context window** — a token is roughly three quarters of a word. The context window is the most tokens one model call can take; the fuller it gets, the slower, costlier, and less accurate the call.
- **Stateless session** — each `pi` session starts with no memory of earlier ones, like a fresh process. Anything that must survive has to be written to a file and read back.
- **Sub-call and lead call** — split a big input into chunks, run one model call per chunk, then one final call over the short results. Map-reduce, applied to prompts.
- `--dry-run` — a flag that makes a script print what it would do without downloading or changing anything.

Tutorial 1 — Set up the team repository and model access

Step 1.1 — Check the prerequisites

Do: Confirm `Node.js >= 22.19`, `Python >= 3.9`, and `gh` logged in via `gh auth login`; then install `pi`:

```
npm install -g @earendil-works/pi-coding-agent
```

Check: `pi` starts from the terminal, and `gh` reports you are logged in.

Step 1.2 — Get your private repo

Do: One team member runs `bash bootstrap.sh` from any directory. It clones the public course repository, creates a private copy named `agentic-engineering-private` under that member's GitHub account (the public fork from Studio 01 is untouched), pushes it, and grants access to the instructor and TAs. That member then adds the other teammates as collaborators under the repository's [Settings > Collaborators](#).

Check: `agentic-engineering-private` exists under that account and every teammate can push to it.

Step 1.3 — Connect a model

Do: Use exactly one of the two supported paths:

1. **Primary:** Set `export OPENAI_API_KEY=...` and use model `openai/gpt-5.6-luna`.
2. **Alternative:** Run `pi`, execute `/login` inside it using a ChatGPT Plus/Pro account, and use model `openai-codex/gpt-5.6-luna`.

Do not use any other provider or path.

Check: the variable is set in the terminal you will run the scripts from, or `pi` shows you logged in.

Step 1.4 — Run setup

Do: From the repository root:

```
cd studio/studio-02/instruction/code
pip install -r requirements.txt
bash setup.sh
```

`setup.sh` downloads BABILong qa1 data (256k/512k/1M buckets, ~700 MB total), builds the 768K bucket, samples 5 items per bucket into a subset benchmark, overrides the agent configuration to raise the model's context window limit, and reports available model providers. Use `bash setup.sh --dry-run` first to preview the plan without downloading or modifying settings.

Check: the dry run prints a plan; the real run finishes and lists your model provider.

Tutorial 2 — Run the three experiments

Run every script from `studio/studio-02/instruction/code` with `--team <team>`. *Note:* All part outputs land under `studio/studio-02/submission/<team>/evidence/part_a/`, `part_b/`, and `part_c/` respectively.

Step 2.1 — Part A: Context Window Stress Test

Do:

```
python3 part_a_stress.py --model openai/gpt-5.6-luna --team <team>
python3 plot_qa1_curve.py --team <team>
```

Why: This tests the model with one call per item at three context sizes (256K, 512K, 768K real input tokens) to observe where performance degrades: single-call attention dilutes as context size increases.

Check: `evidence/part_a/` contains `summary.md`, `results.json`, `results.csv` (from `part_a_stress.py`) and `qa1_curve.png` (from `plot_qa1_curve.py`).

Step 2.2 — Part B: Context Isolation and Targeted Summarization

Do:

```
python3 part_b_isolate_compress.py --model openai/gpt-5.6-luna --team <team>
```

Why: This splits each long item into roughly 96K-token chunks and compares two mitigation strategies: Isolate (one sub-call per chunk, then a lead call over the short reports) and Targeted summary (one ~200-token summary per chunk, then one answer call over the summaries). Reducing peak single-call context tokens bypasses long-context degradation.

Check: `evidence/part_b/` contains `summary.md`, `results.json`, `results.csv`, and `comparison.png`.

Step 2.3 — Part C: File-Based Memory Architecture

Do:

```
python3 part_c_memory.py --model openai/gpt-5.6-luna --team <team>
```

Why: This runs three separate `pi` sessions where the agent must keep a `decisions.md` memory file (per instructions in `part_c/AGENTS.md`); the third session must recall decisions from the first two. File-based memory maintains architectural context across independent stateless runs, compared against a memoryless baseline.

Check: `evidence/part_c/` contains `summary.md` and `results.json`.

Tutorial 3 — Write up and submit

Step 3.1 — Write the explanation

Do: Copy the `submission/_template/EXPLANATION.md` template into your team's submission folder and fill it in. Your document must contain exactly these six sections:

- What we ran
- What degraded in Part A
- What fixed it in Part B
- What memory got right/wrong in Part C
- What remains unknown
- Who did what

Check: the file has exactly those six section headings, each filled in.

Step 3.2 — Submit

Do: One submission per team. Commit and push all changes to your team's private repository before Session 3, Friday, September 25, 2026. Paste your repository URL once on CourseWorks (<https://courseworks2.columbia.edu/courses/251648>) — later studios are collected from the same repository, so you only paste the URL once. Never commit API keys.

Check: the push is visible on GitHub and the URL is posted on CourseWorks.

Wrap-up

Grading: refer to `GRADING_RUBRIC.md` in the same folder; the 10 points are split 6 (completion, 2 per part) + 4 (explanation).

Errata

Block	Source says	Should be	Evidence	Confidence
S11	<code>>= 3.9</code>	Python 3.11+	<code>docs/course-prep.md</code> says “Python 3.11+”, as does the syllabus Resources section; the lab’s <code>setup.sh</code> checks for 3.9. Using 3.11+ satisfies both.	likely

Omitted

Nothing omitted.